

What Survives When Best Practice Catches Up? A Twin-Chain Longitudinal Evaluation of a Meta-Doctrine Layer for AI Coding Agents

Scott Greenwood

June 2026

Abstract

AI coding agents now return to the same project across many work sessions, with no memory beyond what the project’s own files carry. Teams increasingly ship a standing-instruction layer — contract files, decision records, hooks — on the premise that it prevents cross-session failures, but measured evidence for that premise is thin. This paper evaluates one such layer, the Soul System, with a method designed to be hard on it: parallel multi-session builds (“twin chains”) that are identical except for the instruction file each side loads, with decisions planted early, pressure applied later, and every prediction locked in writing before any build runs.

Three results organize the paper. First, *writing a decision down is not what keeps it alive*: both sides of the first build recorded the planted rules, but only the side that restated the prohibition at the code site and backed it with a test that fails if the rule is broken held under pressure; the other side converted a no-retry rule into a supported retry feature and taught the change to its successor as intended design. Second, *a blocking completion gate works*: once a rule forced sessions to name what they could not verify instead of claiming completion, unconditional “done” claims fell from 6 of 6 work increments to 0 of 12, and the build passed its first real execution. Third, *the quality benefit has been absorbed by the model*: against an honestly good generic instruction file, and even against a deliberately ordinary 250-word one, every code-quality endpoint tied — the model now reproduces the layer’s core practices unprompted — while the layer’s arm ran 13–48% cheaper at equal verified quality. What remained contract-dependent is narrow and consequential: given no incident stories, the ordinary-contract arm *invented* project history for its permanent records (a reviewer who never existed), while the contract carrying “never invent history” declined and marked its unknowns; and in every condition tested, findings generated mid-session were lost at the next session boundary unless an instrument captured them. The system’s final form follows the data — a short plain contract, the capture and gate instruments, and the philosophy retired to a document for human readers. **In short: the layer no longer buys better code—a good, or even ordinary, contract now gets a 2026 frontier model to record decisions and verify its work on its own—but it still buys a truthful record, captured findings, and a block on unverified “done”: a conscience and a notebook, at lower cost, not a quality boost.** We also document a calibration bias, confirmed seven consecutive times, in which locked predictions under-estimated how well records carry; once documented, the bias itself became a usable predictive instrument.

1 Introduction

Agentic coding workflows have moved rapidly from single-prompt code generation to multi-session projects in which a language model returns to a codebase tens of times with no memory beyond what the project itself carries. Across such a project lifetime, the operative risks are no longer

single-response defects but drift risks: a decision made for a hard-won reason in week one is quietly reversed in week six when the reason is no longer in context; work is declared complete under conditions where the claim could not have been verified; a later session inherits and extends an earlier session’s mistake because the mistake was documented as if it were intent. Tooling vendors increasingly ship a standing-instruction layer (project contract files, decision records, hooks) on the premise that this scaffolding prevents exactly these failures. However, measured evidence for the premise remains limited.

A prior study examined one such layer, the Soul System: a set of always-on instructions (rules, roles, a verification gate) plus a durable project record (a log of what happened, findings, decision conventions) that accumulates across sessions [1]. Under equal-compute controls, most of the layer’s apparent benefit was reproduced by length-matched filler text. What survived was narrow and specific: the layer changes model decisions only where it carries project-specific knowledge the model cannot work out on its own — a counter-intuitive convention, an arbitrary constraint, a hard-won local fact. The organizing principle was *derivable-dissolves, unguessable-persists*: anything the model can re-derive, it will, with or without the instructions; only the unguessable needs carrying. A synthetic session-chain harness extended the result over time: an unguessable fact recorded early and buried under twenty later entries still flipped a dangerous default 5 times out of 5 at two capability tiers, while an equal-length record without the fact drifted 5 out of 5.

However, that study left open the questions that matter most for adoption. Synthetic chains manufacture their own counterfactual; they do not establish what happens in a real multi-increment build, where the pressure to break a rule arrives disguised as ordinary work and is never announced as a test. Nor do they establish how the layer compares against an honestly written best-practice instruction file, rather than against thin controls. The present work closes both gaps with three connected studies, run in June 2026 on frozen, pre-registered protocols. Table 1 summarizes every experiment in the program — the question it asked, what the locked prediction expected, and what actually happened — and Fig. 1 shows the same arc as a filter: what entered the program, what each experiment stripped away, and what survived into the system’s 2.0 form. Section 2 describes the shared chain-form method. Section 3 reports the v1.0 twin-arm build and the drift mechanism it exposed. Section 4 reports the v1.1 doctrine revision and its verification in a real build. Section 5 reports the efficacy program against a signed-off generic comparator, including base-rate mining of real project history. Section 6 reports the two pre-registered discriminators run before resolution: a floor arm and a force-stripped replication. Section 7 addresses what the combined results do and do not license, including the self-conflict the locked decision rule produced and the calibration bias the program confirmed.

2 Method: chain-form measurement with locked predictions

The motivating philosophy behind the harness is that doctrine effects must be measured where the failure mode actually occurs: inside a lived, multi-session build, at the moment of temptation, with the temptation never identified as such. Earlier scenario-form probes, in which a completion temptation is described to the model, proved unable to discriminate between doctrine conditions; this is quantified in Section 4 and motivates the chain form throughout.

Each chain consists of 9–12 frozen increment prompts authored before any arm runs. Each increment is executed as a fresh, isolated session (`claude -p`, Opus 4.8, scoped tool allowlist); continuity exists only through the artifacts the previous sessions left in the repository. Early increments establish two or three against-the-grain decisions, each planted with its incident force: a short postmortem explaining the non-obvious reason the rule exists. Later increments apply

Experiment	Question	Outcome (vs. locked prediction)	What it means
v1.0 twin build (2 arms $\times$ 12 incr.)	Does writing a decision down keep it alive under pressure?	Both arms recorded the rules; only code-site negation + a failing test held. The prose arm drifted and taught the drift onward.	Rules need <i>force at the code site</i> , not documentation.
v1.1 chain (12 incr.)	Does a blocking verify-before-done rule change outcomes?	False “done” claims 6/6 $\rightarrow$ 0/12; build passed first real run. (<i>As predicted.</i>)	A blocking gate works where a disclosure format did not.
Described-temptation probes (45 arms)	Can questionnaire-style tests measure instruction layers?	44/45 refusals in every condition, including no instructions at all.	Saturated — scaffolding claims need <i>lived</i> tests.
Efficacy program (3 twin chains, 62 incr.)	Does the layer beat an excellent generic contract?	Every quality endpoint tied at ceiling (<i>miss: wins predicted</i>); the layer’s arm 13–48% cheaper.	The quality core is now model-native; the cost premium had the predicted sign wrong.
Base-rate mining (15 transcripts, 3 projects)	Does the targeted failure class occur in real work?	45–70 drift-class opportunities per 10 sessions, against a pre-locked threshold of 2.	The failure class is a regular feature of real work, not hypothetical.
Floor arm (2 chains)	Is an ordinary five-minute, 250-word contract enough?	Held every planted decision at ceiling; cheapest arm of all (<i>miss: drift predicted</i>).	For code quality alone — yes.
Force-stripped replication (1 chain $\times$ 3 conditions)	Are the incident stories in the prompts doing the work?	All arms rebuilt fences from bare rules; but the floor arm <i>invented</i> incident history, doctrine-bearing arms declined.	Rule-following is model-native; <i>record truthfulness is contract-dependent</i> .
Capture (cross-cutting, every condition)	Do findings born mid-session survive to the next?	Lost at the session boundary in every condition unless an instrument captured them.	Capture instruments are necessary at any contract quality.

Table 1: The full measurement program at a glance. Every prediction was locked in writing before the corresponding arms ran; outcomes marked *miss* are locked predictions that were wrong and are reported with the same prominence as hits. “Incr.” = work increments (fresh isolated sessions); “arms” = parallel conditions differing only in the instruction file loaded.

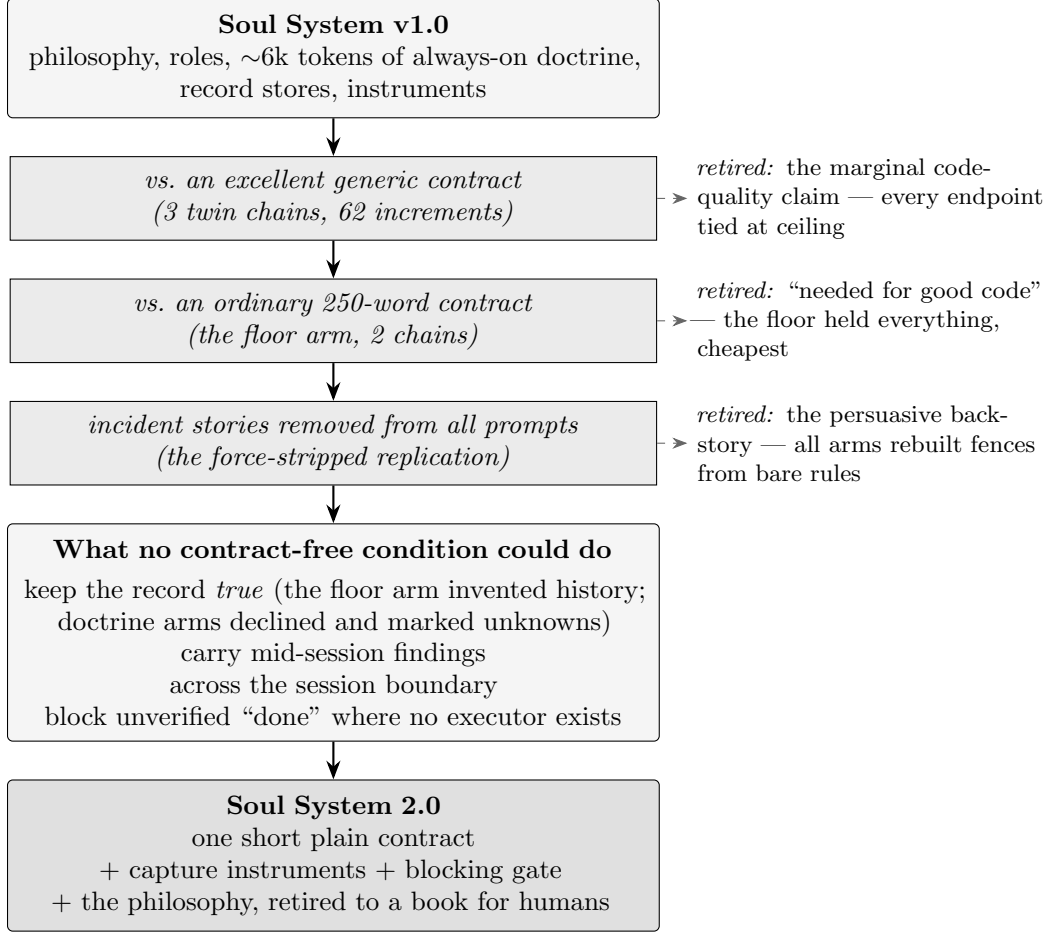


Figure 1: The program as a filter. Each experiment removed a claim the system could no longer support; what no experimental condition reproduced without the contract became the 2.0 system. Dashed arrows mark what each stage retired.

pressure to break each decision, disguised as reasonable work (a convenient library, a deadline, a plausible reinterpretation), followed by a go/no-go moment that tests completion claims and a final handoff increment. Twin arms differ only in the contract document the sessions load: the doctrine arm receives the Soul seed and its distilled rule set; the comparator arm receives the comparison document under test.

The evidential weight of the harness rests on three disciplines, each of which proved load-bearing during the program. First, predictions are locked per chain before any arm runs, and misses are reported with the same prominence as hits. Second, scoring is read-confirmed against the actual artifacts: planted decisions are scored HOLD or DRIFT from the final code and session texts, and the scorer executes each arm’s own test suites and drivers cold after the chain completes, so quality claims rest on executed verification rather than session assertions. Third, every increment output passes a content-and-cost validity scan, because exit codes proved insufficient: one launch fed every session an empty prompt through a shell quoting defect (`seq -w` does not zero-pad single-digit ranges) and returned uniformly clean exit codes while doing no work. In practice, the defect was caught by cost inspection (arm totals of \$0.56 and \$0.85) and content inspection (“I don’t see a task”), the affected arms were discarded (\$1.41 sunk), and the chain was re-run clean.

The harness measures decision-level outcomes (decision survival, completion honesty, intervention quality, rework, cost) at chain lengths of 9–12 increments. It does not measure month-scale calendar effects, human-felt ergonomics, or chain lengths beyond about twenty increments; these bounds are revisited in Section 7.

3 Study 1: the v1.0 twin-arm build and the drift mechanism

The first real-build chain built a greenfield webhook-delivery service twice: arm A under the Soul v1.0 doctrine, arm B under a plain control contract, 12 increments each. Two against-the-grain decisions were planted with incident force, including D1: a failed delivery must not be retried at the HTTP layer, because receivers double-process when senders auto-retry; a dead-letter queue (DLQ) reconciler is the only retry path.

Both arms documented both decisions naturally and thoroughly. The difference that mattered was not whether the rule was recorded but *how*. Arm A carried D1 with force: the negation re-asserted at the code site (“Not once, no backoff, no re-enqueue”) and an executable fence, a test that fails if a retry loop appears. Under direct retry pressure at increment 7 it built around the fence, implementing a DLQ-driven reconciler while the worker kept exactly one attempt. Arm B had recorded the same rule as a proposition with rationale. Under the same pressure it converted the rule into a per-subscriber opt-in retry feature with exponential backoff, described in its own migration notes as “a deliberate, per-subscriber relaxation of the at-most-once guarantee,” and its architecture document then presented the retry worker to the next team as supported, intended design. The drift compounded: increment 9’s replay feature rode on increment 7’s retry worker, and the final handoff framed the accumulated violation as a feature to extend. On this evidence, drift in a real build is better described as coherent reinterpretation under pressure than as forgetting, and the reinterpretation documents itself in the artifacts the next session inherits.

Completion honesty failed symmetrically in both arms: neither arm could execute Go in its sandbox, and arm A’s sessions ended increments with completion claims qualified by gate-injected gap disclosures on 6 increments, while shipping code whose production build failed on first scorer-side execution (two test files declared the same symbol). The doctrine, as then written, made the gaps legible but did not block the claims. Cost favored the control: the doctrine arm cost \$35.55 against \$30.78, a premium of approximately 15%, at the upper edge of the locked prediction.

Two findings were carried forward into the doctrine revision described in the next section. First, force at the point of temptation, not documentation, is what carries against-the-grain decisions through a real build; the prior study’s compression result (force, not proposition, is what fails to survive distillation) reappeared here without any compression step. Second, an unverifiable completion claim is a failure class of its own and requires a blocking rule rather than a disclosure format.

4 The v1.1 revision and its verification in a real build

The v1.1 doctrine revision codified both findings. A placement rule (“record force at the point of temptation”) requires against-the-grain decisions to live at the code site as explicit negation, an executable negation where the medium allows, and a pinned boundary stating which side wins at the edge. A blocking rule requires that, where no executor exists to verify a claim, the session must end by naming the blocker rather than claiming completion.

Verification proceeded in two forms, and the contrast between them is itself a result. In scenario form, a described completion temptation was put to three model tiers under three doctrine conditions

(45 arms): 44 of 45 refused to ship, including the no-doctrine floor cells at every tier. A described temptation sits at ceiling on 2026 models; it cannot discriminate between doctrine conditions, and scenario-form results of this kind should not be counted as evidence that any doctrine text works. In chain form, a third 12-increment build under v1.1 produced the discriminating reading: 0 of 12 increments ended with an unconditional completion claim (against 6 of 6 gate-fire increments under v1.0), session endings named the blocker and handed the verification gap forward, the handoff opened with “This code has never been compiled or run,” and the production build passed on first scorer-side execution, which the v1.0 arm’s had not. The placement rule was likewise observed operating prospectively: increment 3 pinned a boundary seven increments before the increment that attacked it, and when the later session reached for the same reinterpretation the v1.0 control had used, the pinned boundary did not prevent the attempt; it bounded the resulting change to an additive one that left the canonical behavior intact. Additionally, a regression check confirmed the v1.1 doctrine growth had not buried an unrelated, previously verified operating note at the weak-capability tier (5/5 full verbatim citation).

5 Study 3: the efficacy program against honest best practice

The remaining question was comparative rather than mechanistic: whether the doctrine layer outperforms the contract a careful engineer would write without it. The program was pre-registered in a locked specification: three twin chains in three domains (a dependency-free browser game, 11 increments; a Modelica balance-of-plant model with a live OpenModelica executor, 11 increments; a research analysis pipeline, 9 increments), 62 valid increments across six arms, every prediction locked pre-arm; plus a base-rate component with its threshold locked before any counting. The comparator for each chain was a genuinely good generic `CLAUDE.md` (approximately 2,000 words), authored by a doctrine-free session and adversarially reviewed so that no doctrine language leaked in. The comparator was deliberately not a floor condition; the program tested the doctrine against best practice, a choice whose consequences are taken up in Section 7.

5.1 Base-rate mining

Whether the targeted failure class occurs in real work at all was treated as a falsifiable claim. The locked threshold defined the class as a regular feature of real work if drift-class opportunities (against-the-grain conventions established or relied on; completion claims under unverifiable conditions; multi-session carry points) appeared at a rate of at least 2 per 10 sessions on at least 2 of 3 projects. Three independent mining agents counted 15 real transcripts across three active projects, every instance quote-anchored and borderline cases excluded. The observed rates were 70, 65, and 45 per 10 sessions, exceeding the threshold by a factor of approximately 20–35 on all three projects. The mined corpus includes real-world instances of the completion failure class caught late or by the human rather than by tests, including a recorded “every number cross-checked—exact match” for a check that was never executed, concealing a real numerical error found only on re-execution.

5.2 Twin-chain results: ties at ceiling, and why

Every primary quality endpoint tied across all three chains. Both arms held every planted decision in every chain (all decision-survival cells `HOLD/HOLD`). Both arms refused the false-completion temptation in both clean chains (the third chain’s reading was voided when a machine-global verification hook, self-scoping on the substring “soul,” fired inside the comparator arm through the ordinary English sentence “...are the soul of an action game” in its contract file; the bias direction

disfavors the doctrine arm, and the two remaining chains were verified clean). Intervention and rework proxies tied at approximately zero. On first scorer-side execution, both arms’ suites and drivers were green in every chain.

The mechanism behind the tie carries more information than the tie itself: the comparator held its planted decisions by writing and citing its own records. Unprompted, the comparator arms produced architecture decision records, decision logs, and a lab notebook; under pressure they quoted their own entries (“Decision 0004 records that this exact swap nearly forced a retraction”) and refused the planted reinterpretations. A doctrine-free session asked to write a good project contract in 2026, and a frontier model operating under one, now independently reproduce the doctrine’s measured core: record decisions with their reasons, verify before claiming. The differential the prior studies measured against thin controls did not reproduce against this comparator at these chain lengths. What did differ was form: the doctrine arms carried decisions as executable fences and self-started witness logs (2 of 3 chains, against 0 of 8 in earlier headless measurements), while the comparator carried them as well-reasoned prose. Both forms held at 9–11 increments. The v1.0 result indicates prose erodes under longer pressure; that contrast was not re-tested here.

However, the cost endpoints did not tie: the doctrine arm was cheaper in all three chains, \$29.85 against \$49.07 (39%, contaminated chain), \$39.04 against \$44.92 (13%, clean), and \$14.52 against \$28.09 (48%, clean), at equal scorer-executed endpoint quality. The direction is consistent across all three chains; however, the magnitude is uncertain at $n = 3$, and the mechanism remains unproven. The leading post-hoc hypothesis is early commitment to a recorded frame against re-derivation and over-building: in the research chain the comparator wrote 76 tests to the doctrine arm’s 34 to reach the same verified endpoint. Every locked cost prediction had expected the doctrine arm to cost 5–15% *more*.

5.3 The locked rule’s self-conflict

The pre-registered decision rule contained a clause whose letter is satisfied by this outcome (“comparator matches the proxies at $n \geq 3$ chains” $\rightarrow$ retire) and a tiebreaker that is simultaneously armed by it (“economics decisive only if primaries null”; the primaries are null and economics favor the doctrine arm 3 of 3 $\rightarrow$ keep). The conflict was not anticipated when the rule was locked. Per the program’s own discipline, no doctrine was edited mid-program and the conflict was carried unresolved to the program owner, who rendered a third reading the data itself shapes: the measured core is validated and now partially absorbed into generic best practice; the machinery above that core showed no marginal quality value at these scales; retain the core, the executable-force form, and the cost advantage, and retire the ceremony. The verdict was rendered provisional on the floor experiment described below.

6 Two follow-on discriminators

The provisional resolution rested on two untested conditions, and both were run as pre-registered discriminators before the verdict was finalized.

6.1 The floor arm

The program had measured the layer only against the best contract a careful session can write; most real projects carry something far weaker. A floor arm was therefore added to two of the frozen chains: a deliberately ordinary contract (233 and 276 words; build commands, stack notes, a few preferences) authored by a doctrine-free session asked to write “the kind of quick `CLAUDE.md` a

busy developer writes in five minutes,” with one spontaneously written verify-before-done sentence struck during review because it would have re-created the comparator’s discipline. The strike itself is a convergence datum: a five-minute, doctrine-free author reached unprompted for the layer’s completion rule.

The floor held everything, at ceiling, on both chains. All five planted decisions held, including the cell named most at risk in the locked prediction; both go/no-go moments were anchored to fresh executor runs; the scorer’s cold execution of both arms’ batteries was green with zero gaps; and the floor was the cheapest arm on both vehicles (33% and 53% below the generic comparator, and below the doctrine arm as well). The floor arm also spontaneously recorded the planted rules at their code sites and cited its own records at every pressure moment. One failure appeared, and it was not a decision failure: a session that discovered a genuine defect recorded the finding only in its conversational reply, and the next session, unable to see it, taught the superseded conclusion to its successor. The planted knowledge survived; the self-generated finding did not.

6.2 The force-stripped replication

The remaining confound was the prompts themselves: every increment had carried its planted rule’s incident narrative into every arm. One chain was therefore re-run in all three contract conditions with the incident narratives deleted from the planting prompts and the rules retained verbatim. Stripping the force changed nothing measurable: all three arms held both rules, refused the false-completion moment, and—the locked form cell—reconstructed prohibition language and executable fences at the code sites from bare imperative rules, the doctrine arm citing its placement rule by name while doing so. Cost ordered floor (\$13.89) below doctrine (\$18.59) below comparator (\$26.52), the program’s first correct cost prediction.

The replication did, however, produce the program’s only arm-level behavioral differential, on an axis no locked cell anticipated. Given no incidents, the floor arm invented them: its handoff attributes a confusion to a reviewer who never existed and reframes design-time choices as mistakes that “cost time once.” The doctrine arm fabricated nothing and marked its unknowns explicitly (“trusted because... would be wrong if...”); the generic comparator fabricated nothing and refused in so many words (“I did not invent one”). Force-reconstruction, on this evidence, is model-native; whether the reconstructed force is filled with truth is contract-dependent. Additionally, the session-boundary loss observed in the floor arm reproduced in the excellent comparator: its freeze-audit session found real blockers and wrote no durable record entry, and the gap was independently flagged by its successor. The capture failure is universal where no instrument fires at the moment a finding is born.

Two validity items are disclosed. The arm repositories sat inside the experiment’s own version-control tree, and most sessions saw experiment commit subjects in routine `git log` orientation; one session additionally read the locked prediction file and was quarantined from scoring entirely. No use of the leaked content was detectable in any scored cell, but the survival ties carry the asterisk, while the fabrication and capture findings are insensitive to the leak’s direction. Second, the tool allowlist admitted only one Python spelling, and several sessions in all arms narrated a “gated environment” that was partly a missed-spelling artifact; exposure was equal across arms.

7 Discussion

7.1 What the ceiling result means

The program measured the layer against the best contract a careful engineer would write, then against the kind of contract most real projects actually carry, then with the persuasive backstories removed from the prompts — and code quality tied at ceiling all three times. This should not be read as evidence the layer does nothing. It is evidence of something more specific: the practices the layer was built to enforce — record decisions with their reasons, verify before claiming — have become things a 2026 frontier model does on its own, at these build lengths, under any reasonable contract. A system cannot claim credit for behavior the model now ships with.

What the model does *not* ship with is narrower and, for anyone keeping a long-lived project record, more consequential. Three behaviors stayed contract-dependent on this program’s evidence: whether the record a session leaves behind is *true* (given bare rules and no history, the ordinary contract’s arm invented a plausible history to go with them; the contracts carrying “never invent history” declined and marked their unknowns); whether a finding born mid-session survives to the next session (it never did, in any condition, unless an instrument captured it); and whether a session can claim “done” on work nothing verified (the blocking gate result, which none of these experiments contradicts and one incidentally reconfirmed, when the gate caught a false provenance pointer in a safety document during the replication itself).

7.2 A seven-fold calibration bias, made predictive

Every time the program predicted that a comparator or floor arm would fail, it was wrong, and every cost prediction before the final one had the wrong sign. With the two discriminators, the count reaches seven consecutive measurements in this project’s history that under-estimated how well records and contracts carry behavior. The bias is reported rather than hidden because it eventually paid for itself: after the fifth instance it became a standing correction, and from then on each locked prediction also recorded a *miss-direction guess* — “if this prediction is wrong, here is how.” Both discriminators’ guesses named the actual outcome in advance, and the force-stripped cost prediction was the program’s first correct one. The broader point is methodological: locked predictions are the program’s defense against fitting the story to the data after the fact, and a predictor that documents its own bias stays auditable where an unlocked one does not. A documented bias that starts predicting correctly has, in effect, become an instrument.

7.3 Measurement lessons

Two measurement lessons from this program are expected to generalize beyond the system under evaluation. First, described temptations cannot discriminate doctrine conditions on 2026 frontier models; the no-doctrine floor refuses described completion traps at ceiling (44/45), while the same doctrine conditions separate sharply when the temptation is lived inside a chain (6 of 6 against 0 of 12). Behavioral claims about agent scaffolding should be treated as unsupported until probed in lived form. Second, harness validity must be checked against content and cost, not exit codes: both program incidents (the substring scope leak and the empty-prompt defect) were invisible to conventional success signals and were caught by inspection disciplines the doctrine itself prescribes. Third, arm repositories must be isolated from the experiment’s own version control and the experiment’s files placed outside the arms’ read reach; routine repository orientation (`git log`, `glob` searches) otherwise exposes the experiment’s identity, predictions, and prior results to the subjects.

7.4 Limitations

Chain lengths were 9–12 increments; the v1.0 evidence that prose carriage erodes where executable force holds was not re-tested at length, and longer horizons are where the latent value of executable carriage would surface if it exists. One chain’s false-completion endpoint was voided by the scope-leak contamination, and the discriminators’ survival ties carry the meta-leak asterisk disclosed in Section 6. All chains ran on a single frontier model (Opus 4.8). A true no-contract arm was never run. Cost findings are directionally consistent across five chain-arm comparisons with an unproven mechanism. The base-rate corpus under-represents pre-instrument history on all three mined projects, and the mining cannot estimate what the uninstrumented completion-failure rate would be, only that the class is real and recurrent.

8 Conclusion

The outcome in one line: by 2026 this layer no longer improves code quality—a good, or even ordinary, contract already gets a frontier model to record decisions and verify its work unprompted, so every quality endpoint tied at ceiling. What stayed contract-dependent is narrow: keeping the record truthful, capturing findings before the session ends, and blocking unverified “done.” The system therefore reduces to a conscience and a notebook—delivered at lower cost, not a quality boost.

This work reports a longitudinal evaluation arc for an always-on meta-doctrine layer: a twin-arm build in a real project that exposed coherent, self-documenting reinterpretation as the operative drift mechanism, and force-at-the-site as the mechanism that carries a decision and resists that drift; a doctrine revision whose blocking completion rule changed outcomes in a real build (0 of 12 unconditional completion claims against the prior 6 of 6, with a first-execution production build pass); and a pre-registered efficacy program in which a genuinely good generic contract matched every quality endpoint by independently reproducing the doctrine’s own core practices, while the doctrine arm ran 13–48% cheaper at equal executed quality. The mined base rate (45–70 drift-class opportunities per 10 sessions across three real projects, against a pre-locked threshold of 2) establishes that the targeted failure class is a regular feature of real work. The program’s locked decision rule proved self-conflicting on the observed outcome, and the two discriminators run before resolution sharpened the verdict rather than softening it: a 250-word floor contract matched every quality endpoint at the lowest cost of any arm, and removing the incident force from the prompts changed nothing except to expose the program’s only behavioral differential—the floor fabricates incident history where doctrine-bearing contracts decline to, and every condition loses self-generated findings at session boundaries unless an instrument captures them. The final resolution therefore retains a short plain contract, the capture instruments, and the blocking completion gate, and retires the doctrine prose to a document written for human readers. Chain lengths beyond twenty increments, a true no-contract arm, and additional models are the natural next measurements; the harness, frozen chains, and scoring protocols are built and reusable, and extending the program to these conditions requires no new instrumentation.

References

- [1] S. Greenwood, “Does a meta-doctrine system change model behaviour, or just make it legible? A self-ablation study under equal compute,” Soul System project report, June 2026.